



Decision Support

Many-Objective Pareto Local Search

Andrzej Jaszekiewicz



Faculty of Computing, Institute of Computing Science, Poznan University of Technology, ul. Piotrowo 2, 60-965 Poznan, Poland

ARTICLE INFO

Article history:

Received 25 July 2017

Accepted 5 June 2018

Available online 15 June 2018

Keywords:

Metaheuristics

Multiobjective optimization

Combinatorial optimization

Pareto Local Search

ABSTRACT

We propose a new Pareto Local Search Algorithm for the many-objective combinatorial optimization. Pareto Local Search proved to be a very effective tool in the case of the bi-objective combinatorial optimization and it was used in a number of the state-of-the-art algorithms for problems of this kind. On the other hand, the standard Pareto Local Search algorithm becomes very inefficient for problems with more than two objectives. We build an effective Many-Objective Pareto Local Search algorithm using three new mechanisms: the efficient update of large Pareto archives with ND-Tree data structure, a new mechanism for the selection of the promising solutions for the neighborhood exploration, and a partial exploration of the neighborhoods. We apply the proposed algorithm to the instances of two different problems, i.e. the traveling salesperson problem and the traveling salesperson problem with profits with up to 5 objectives showing high effectiveness of the proposed algorithm.

© 2018 Published by Elsevier B.V.

1. Introduction

Combinatorial optimization problems find numerous applications in transportation, logistics, scheduling, design, etc. (Yu, 2013). On the other hand, real-life optimization problems often require taking into account multiple points of view corresponding to multiple objectives. Since, multiple objective combinatorial optimization problems are usually NP-Hard, multiobjective metaheuristics are often used to solve them approximately (Talbi, Basseur, Nebro, & Alba, 2012).

Effective single objective algorithms for combinatorial optimization problems usually use some kind of local search as one of their components. Pareto Local Search (PLS) (Angel, Bampis, & Gourvès, 2004; Paquete, Schiavinotto, & Stuetzle, 2007; Paquete & Stuetzle, 2006) is a very natural extension of the concept of the local search to the multiobjective case. PLS works with a Pareto archive, i.e. a set of potentially Pareto-optimal solutions that are not dominated by any other solution obtained so far. PLS explores neighborhood of each solution in the archive and uses the neighbor solutions to update the Pareto archive.

Alike the single objective local search, PLS when used as a standalone algorithm started with an initial archive of a low quality is not very effective, however, it is an important component of a number of the state-of-the-art hybrid algorithms e.g. for the bi-objective knapsack (Lust & Teghem, 2012), the bi-objective traveling salesperson problem (TSP) (Cornu, Cazenave, & Vanderpooten, 2017; Jaszekiewicz & Lust, 2017; Liangjun, Qingfu, & Battiti,

2014; Lust & Jaszekiewicz, 2010; Lust & Teghem, 2010), various bi-objective permutation flowshop problems (Dubois-Lacoste, López-Ibáñez, & Stützle, 2011), the bi-objective set covering problem (Lust & Tuytens, 2014), and the risk-cost optimization for procurement planning (Mori, Kobayashi, Samejima, & Komoda, 2017).

The effectiveness of PLS as a component of the hybrid algorithms may be explained by the fact that it has different characteristics than most other multiobjective metaheuristics. In order to obtain a good approximation of the Pareto front each multiobjective metaheuristic should search both towards and along this front. The search towards the Pareto front means generating new solutions lying closer to the Pareto front than known solutions, and the search along the Pareto front means generating new potentially Pareto-optimal solutions improving the representation of the Pareto front. PLS is very effective in the search along Pareto front when started from a seed of high quality solutions. For example in the case of the bi-objective traveling salesperson problem such a seed may be generated with the use of the Lin–Kernighan (Lin & Kernighan, 1973) heuristic (Cornu et al., 2017; Liangjun et al., 2014; Lust & Jaszekiewicz, 2010; Lust & Teghem, 2010).

The standard PLS algorithm becomes, however, very inefficient in the case of more than two objectives because the number of Pareto-optimal solutions grows very fast with the number of objectives. Thus, PLS has to search neighborhoods of a huge number of solutions and the process of updating large Pareto archives with new solutions becomes very time-consuming. Indeed, with very few exceptions (e.g. Cornu et al., 2017; Liangjun et al., 2014), majority of the publications on PLS concerns bi-objective optimization only.

E-mail address: andrzej.jaszekiewicz@cs.put.poznan.pl

In this paper we propose a new Many-objective PLS algorithm (MPLS) dedicated to the problems with more than two objectives. Please note, that we use the term “Many-objective” in somehow non-standard way, since it usually describes problems with ≥ 4 objectives (Chand & Wagner, 2015). However, in the context of PLS, three objectives is already a high number.

The proposed algorithm is based on the following three new mechanisms that differ it from the standard PLS:

- The use of a recently proposed ND-Tree data structure (Jaszkiwicz & Lust, 2018) for the efficient update of the Pareto archive.
- The use of a new mechanism for the selection of the promising solutions for the exploration of their neighborhoods based on the randomly selected weighted Chebycheff scalarizing functions.
- The partial exploration of the neighborhoods, i.e. testing only some of the neighborhood solutions.

The first mechanism reduces the time needed to update the Pareto archive, while the two latter mechanisms improve the quality of the archive with a limited number of the neighborhood solutions tested.

We apply MPLS to two different multiobjective combinatorial optimization problems: the multiobjective traveling salesperson problem (MTSP) and the multiobjective traveling salesperson problem with profits (MTSPWP) with up to 5 objectives. Through a computational study we show that MPLS produces archives of much better quality than the standard PLS in the same time. We also show that each of the three new mechanisms described above contributes to the effectiveness of the proposed method. Furthermore, since for the bi-objective traveling salesperson problem the best results have been obtained by properly balancing the CPU time assigned to the Lin-Kernighan heuristic and to PLS (Cornu et al., 2017; Jaszkiwicz & Lust, 2017; Liangjun et al., 2014; Lust & Jaszkiwicz, 2010; Lust & Teghem, 2010) we test whether a similar situation holds in the case of MTSP with more than two objectives.

The paper is organized in the following way. In the next section we describe the proposed Many-Objective Pareto Local Search algorithm. Then the computational experiment is presented. In Section 4, related works are discussed. Finally, conclusions and directions for further research are presented.

2. Many-Objective Pareto Local Search algorithm

We start the description of the proposed MPLS algorithm with a short presentation of the standard PLS that is a basis for our algorithm. Then we describe the three new mechanisms used in MPLS. Finally, we present the whole algorithm of MPLS.

2.1. Pareto Local Search

PLS is a relatively straightforward adaptation of the idea of the single objective local search to the multiobjective case. In PLS the acceptance of a new neighborhood solution is based on the dominance relation.

Consider a general *multiobjective optimization problem* with a feasible set $\mathcal{X}$ and d objective functions $y_k(x)$. Without loss of generality we assume that the objectives are maximized. The image of a feasible solution x in the objective space $\mathbb{R}^d$ is a point $y(x) = (y_1(x), y_2(x), \dots, y_d(x))$. We say that a point $y^1 \in \mathbb{R}^d$ dominates a point $y^2 \in \mathbb{R}^d$ if, and only if, $y_j^1 \geq y_j^2 \forall j \in \{1, \dots, d\} \wedge \exists j \in \{1, \dots, d\} : y_j^1 > y_j^2$. We denote this relation by $y^1 \succ y^2$. For the sake of simplicity we will use the dominance relation with respect to the corresponding solutions as well. The solutions not dominated

by any other feasible solution are called *Pareto-optimal* and the image of the set of Pareto-optimal solutions in the objective space is the *Pareto front*.

A set of distinct and mutually non-dominated solutions, i.e. solutions such that none of them dominates any other, is called a *Pareto archive*. In the context of multiobjective metaheuristics the Pareto archive is used to store the set of potentially Pareto-optimal solutions generated by the method till a given iteration.

PLS works with a Pareto archive. It starts with some initial Pareto archive and then searches whole neighborhoods of all solutions whose neighborhoods have not been searched yet for new potentially Pareto-optimal solutions until no such new solution can be found in any of the neighborhoods (see Algorithm 1 based on

Algorithm 1 PLS.

Parameter $\downarrow$: $\mathcal{A}$: an initial Pareto archive

```

 $P \leftarrow \mathcal{A}$ 
 $P_a \leftarrow \emptyset$ 
while  $P \neq \emptyset$  do
  for all  $x \in P$  do
    for all  $x' \in \mathcal{N}(x)$  do
      if  $y(x) \neq y(x')$  and  $y(x) \neq y(x')$  then
        if  $\text{Update}(\mathcal{A} \downarrow, x' \downarrow)$  then
           $P_a \leftarrow P_a \cup \{x'\}$ 
        end if
      end if
    end for
  end for
   $P \leftarrow P_a$ 
   $P_a \leftarrow \emptyset$ 
end while

```

where $\mathcal{N}(x)$ denotes the neighborhood of x and $\text{Update}()$ updates the Pareto archive $\mathcal{A}$.

the particular version of PLS used e.g. in Lust & Teghem, 2010).

2.2. Mechanism I: efficient update of the Pareto archive with the use of ND-Tree

Whenever a new neighbor solution not dominated by the current solution is found, the Pareto archive needs to be updated. Updating a Pareto archive $\mathcal{A}$ with a new solution x' means that:

- x' is added to $\mathcal{A}$ if it is not dominated nor equal to any solution in $\mathcal{A}$,
- all solutions dominated by x' are removed from $\mathcal{A}$.

Since the neighborhood solutions for many combinatorial problems may be generated in a very short time and the number of Pareto-optimal solutions grows fast with growing number of objectives, the process of updating the Pareto archive may easily become the main factor influencing the running time of PLS.

The simplest approach to the update of the Pareto archive is to organize it as a simple list of solutions and compare the new solution to each solution in this list until a dominating or equal solution has been found or all solutions have been compared. This approach is, however, very time consuming for large Pareto archives. In the bi-objective case the Pareto archive may be efficiently updated using the binary search in an archive sorted according to the values of the objectives (Jaszkiwicz & Lust, 2018). This approach, however, cannot be extended to the case of more than two objectives.

In MPLS we use a recently proposed ND-Tree data structure (Jaszkiwicz & Lust, 2018). In ND-Tree the Pareto archive is recursively split into disjoint subsets. For each subset approximate local

ideal and approximate local nadir points, i.e. points in the objective space respectively dominating and dominated by all solutions in this subset, are kept. In other words, all solutions from each subset are contained in the hyper-box defined by the two points. These hyper-boxes may in general overlap, but the tree is built such that the subsets contain close solutions contained in small hyper-boxes. By comparing the new solution to the approximate local ideal and the approximate local nadir points many branches of the tree can be omitted. The computational complexity of updating the Pareto archive with ND-Tree is sub-linear with respect to the size of the archive for any number of objectives under mild assumptions (Jaskiewicz & Lust, 2018). Thus the overall running time of MPLS can be significantly reduced with the use of this data structure.

2.3. Mechanism II: selection of the promising solutions for the exploration of their neighborhoods

The standard PLS algorithm ends-up with a Pareto-archive being locally optimal, i.e. when no solution being a neighbor of any solution in the archive can improve this archive. Obtaining a locally optimal Pareto-archive is, however, usually prohibitively time-consuming in the case of three and more objectives even with the use of ND-Tree. On the other hand, the algorithm when stopped earlier may produce an archive of a low quality, because there could be high variations in the CPU times spent in exploring various regions of the Pareto front. Dubois-Lacoste et al. (2011) and Dubois-Lacoste, López-Ibáñez, and Stützle (2015) proposed anytime PLS algorithm for the bi-objective case whose goal is to generate archives of a good quality at any iteration. One of the main mechanisms of that algorithm is the selection of solutions whose neighborhood exploration has a high potential to improve the current archive. Dubois-Lacoste et al. use the so-called optimistic hypervolume to select such solutions in the bi-objective case. This mechanism, however, cannot be directly extended to the case of more than two objectives. Thus, we propose a new selection mechanism described below.

In each iteration we would like to generate a new solution highly improving the quality of the Pareto archive. Since the neighbor solutions are usually similar to the original solutions, it is natural to expect that new good solutions may be found in the neighborhoods of known good solutions.

Assume that an indicator of the quality of the Pareto archive is used. A good solution is a solution that substantially contributes to the value of this indicator, i.e. its removal would substantially deteriorate the value of this indicator. A number of indicators for the evaluation of the quality of Pareto archives have been proposed. An often used quality indicator is the hypervolume of the space dominated by the set of points (see e.g. Zitzler, Thiele, Laumanns, Fonseca, & da Fonseca, 2003). Our approach is motivated by another quality indicator – the expected value of the weighted Chebycheff scalarizing functions (Jaskiewicz, 2002). Such functions are defined in the following way:

$$s_{\infty}(y(x), y^0, \Lambda) = \max_{k=1, \dots, d} \{\lambda_k (y_k^0 - y_k(x))\}$$

where y^0 is a reference point, $\Lambda = [\lambda_1, \dots, \lambda_d]$ is a weight vector such that $\lambda_k \geq 0 \forall k$. Each weighted Chebycheff scalarizing function has at least one global minimum belonging to the set of Pareto-optimal solutions. For each Pareto-optimal solution x there exists a weighted Chebycheff scalarizing function s_{∞} such that x is a global minimum of s_{∞} (see Steuer, 1986, ch. 14.8).

The quality indicator is defined as:

$$R(\mathcal{A}, \mathcal{L}) = \mathbb{E}_{\Lambda \in \mathcal{L}} (\min_{y \in \mathcal{A}} s_{\infty}(y, y^0, \Lambda))$$

where $\mathcal{L}$ is a set of all weight vectors.

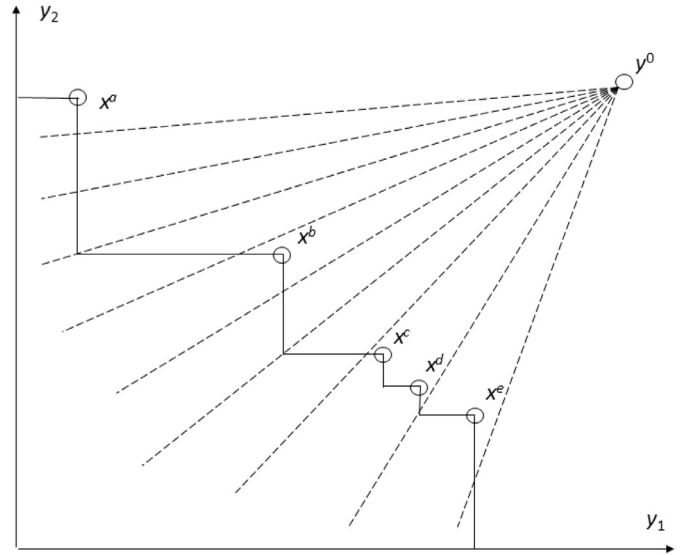


Fig. 1. Comparison of the hypervolume and the expected value of the weighted Chebycheff scalarizing functions indicators.

Although the two indicators, i.e. the hypervolume and the expected value of the weighted Chebycheff scalarizing functions, are defined in quite different terms, they are in fact closely related, which is illustrated in Fig. 1. Consider the non-dominated border of the dominated area used in the hypervolume indicator. Each point lying on this border is optimum of one or more weighted Chebycheff scalarizing functions and has the same values of these scalarizing functions as some of the solutions in the archive. In other words, the hyper-facets of this border belong to the same isoquants of multiple Chebycheff functions as some solutions from the archive. In result, any improvement of the hypervolume implies an improvement of the best values of some weighted Chebycheff scalarizing functions.

An important advantage of the expected value of the weighted Chebycheff scalarizing functions indicator is that to select good solutions we do not need to explicitly calculate the contribution of each solution to this quality indicator. Instead in each iteration we draw a random weight vector defining a weighted Chebycheff scalarizing function and select from the Pareto archive the solution that minimizes the value of this function. Thus, solutions being minima of many scalarizing functions, and so having high impact on the expected value, have higher chance to be selected. For example, in Fig. 1 solutions x^a and x^b have higher chance to be selected than solutions x^c , x^d , and x^e .

This mechanism not only selects promising solutions but also assures a uniform coverage of all regions of the objective space. If the solutions were selected at random with uniform probability, regions that have more dense representation in the current archive would have a higher chance to be explored further. Thus, any random differences in the density would tend to be further increased, since the dense regions would be explored more than regions with a low density of solutions.

Naive approach for finding the solution minimizing a given scalarizing function through evaluation of each solution would be very time consuming. However, ND-Tree data structure used to update the Pareto archive may also be used to efficiently find the solution with the best value using Algorithm 2 proposed in this paper. Let us remind that in ND-Tree the Pareto archive is recursively split into disjoint subsets associated with approximate local ideal points. The algorithm is based on the observation that the value of each weighted Chebycheff scalarizing function for any solution in a subset is not lower than its value for the approximate local ideal

Algorithm 2 MinimizeChebycheffScalarizingFunction.

Parameter $\downarrow$: A node n
 Parameter $\downarrow$: A weighted Chebycheff scalarizing function $s_\infty(\dots, \Lambda)$
 Parameter $\uparrow$: Minimum value of the scalarizing function known so far s^*
 Parameter $\uparrow$: Solution x minimizing the scalarizing function

```

if  $n$  is a leaf node then
  Calculate the value of  $s_\infty(\dots, \Lambda)$  for each solution in the subset
  if the best value is better than  $s^*$  then
    Return the best solution and update  $s^*$ 
  end if
else
  Select the sub-node  $n'$  with the minimum value of  $s_\infty(\hat{z}^*(S(n')), \dots, \Lambda)$ 
  if  $s_\infty(\hat{z}^*(S(n')), \dots, \Lambda) < s^*$  then
    MinimizeChebycheffScalarizingFunction ( $n'$ ,  $s_\infty(\dots, \Lambda)$ ,  $s^*$ ,  $x$ )
    for all Remaining sub-nodes  $n''$  do
      if  $s_\infty(\hat{z}^*(S(n'')), \dots, \Lambda) < s^*$  then
        MinimizeChebycheffScalarizingFunction ( $n''$ ,  $s_\infty(\dots, \Lambda)$ ,  $s^*$ ,  $x$ )
      end if
    end for
  end if
end if

```

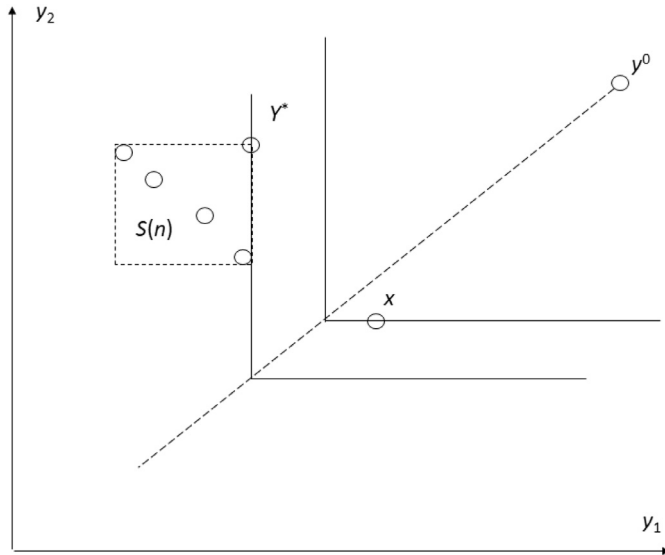


Fig. 2. Set $S(n)$ that could be omitted due to the knowledge of the scalarizing function value for solution x .

point of this subset. Thus, if a value of this function for a solution is already known (upper bound) and this value is not worse than the value for the approximate local ideal point, the whole subset could be omitted (see Fig. 2). If this subset corresponds to an internal node, its whole subtree could be omitted. To find quickly a solution with a good value of the scalarizing function defining a good upper bound at each internal node we select first the sub-node with the best value for its approximate local ideal point.

Since Algorithm 2 is a new contribution reported in this paper, we analyze the computational complexity of this algorithm. This analysis is similar to the analysis of the algorithm for updating the Pareto archive with ND-Tree presented in Jaskiewicz and

Lust (2018). We start by formally defining the ND-Tree data structure :

Definition 1. ND-Tree data structure is a tree with the following properties:

1. With each node n is associated a set of solutions $S(n)$.
2. Each leaf node contains a list $\mathcal{L}(n)$ of solutions and $S(n) = \mathcal{L}(n)$.
3. For each internal node n , $S(n)$ is the union of disjoint sets associated with all sub-nodes of n .
4. Each node n stores an approximate ideal point $\hat{y}^*(S(n))$ such that $\hat{y}^*(S(n)) \geq x \quad \forall x \in S(n)$ and an approximate nadir point $\hat{y}_*(S(n))$ such that $x \geq \hat{y}_*(S(n)) \quad \forall x \in S(n)$.
5. If n' is a sub-node of n , then $\hat{y}^*(S(n)) \geq \hat{y}^*(S(n'))$ and $\hat{y}_*(S(n')) \geq \hat{y}_*(S(n))$.

In the worst case, at each intermediate node we need to analyze each sub-node. Thus, all solutions will need to be evaluated and the computational complexity of Algorithm 2 is $\Theta(N)$ where N is the number of solutions in the archive.

In the best case, at each intermediate node only one sub-node has to be analyzed and the solutions are equally split into a predefined number of sub-nodes. In this case the number of evaluated solutions and points is described by the following recurrence:

$$T(N) = 2 + T(N/C) = \Theta(\log_C N) \quad (1)$$

where C is the number of sub-nodes. Please note, that it is possible to consider even more optimistic scenarios, e.g. when only one solution needs to be evaluated, however, the scenario analyzed above is much more realistic.

The most interesting but also the most difficult to analyze or even define is the average case. Consider the case when each internal node has exactly two sub-nodes. Assume that the number of solutions in one of the sub-nodes is drawn from the uniform distribution, i.e. each number of solutions is equally likely. The other sub-node will contain the remaining number of solutions. In an internal node, either one or both sub-nodes will be processed. Assume that the probability p_2 of processing both nodes is constant. In this case:

$$T(N) = 2 + \frac{1}{N} (1 + p_2) \sum_{k=1}^{N-1} T(k) \quad (2)$$

Multiply both sides by N :

$$NT(N) = 2N + (1 + p_2) \sum_{k=1}^{N-1} T(k) \quad (3)$$

Assume that $N \geq 2$:

$$(N-1)T(N-1) = 2(N-1) + (1 + p_2) \sum_{k=1}^{N-2} T(k) \quad (4)$$

Subtract Eqs. (3) and (4):

$$\begin{aligned} NT(N) - (N-1)T(N-1) &= \\ 2N - 2(N-1) + (1 + p_2) \left(\sum_{k=1}^{N-1} T(k) - \sum_{k=1}^{N-2} T(k) \right) &\Rightarrow \\ T(N) &= \frac{2}{N} + \frac{N+p_2}{N} T(N-1) \end{aligned} \quad (5)$$

The above recurrence has the following solution:

$$T(N) = \frac{\Gamma(p_2 + N + 1)}{\Gamma(N + 1)} - \frac{2}{p_2} \quad (6)$$

where Γ denotes the Gamma function being an extension of the factorial to real number arguments.

Since

$$\lim_{N \rightarrow \infty} \frac{\Gamma(N + \alpha)}{\Gamma(N)N^\alpha} = 1 \quad (7)$$

$\Rightarrow$

$$T(N) = \Theta(N^{p_2}) \quad (8)$$

Algorithm 2 is sub-linear for any $p_2 < 1$ under the above assumptions. Of course, it is still a very simplified analysis of the practical behavior of the algorithm. For example, the probability p_2 will not, in general, be constant. In fact, it may happen that none of the sub-nodes will need to be processed, since the ideal points of all sub-nodes will have the values of the scalarizing function above the current upper bound. Such situations, would further improve the practical efficiency of the algorithm.

2.4. Mechanism III: partial exploration of the neighborhoods

In the standard PLS the whole neighborhood of each solution is explored. This strategy is not well adapted, however, to the goal of obtaining a good anytime behavior since it may lead to the situations where some regions of the Pareto front will be intensively explored while other regions will be underexplored. Dubois-Lacoste et al. (2011, 2015) and Liefoghe, Humeau, Mesmoudi, Jourdan, and Talbi (2012) observed that the anytime behavior of PLS can be improved by a partial exploration of the neighborhoods, i.e. testing only some neighbor solutions.

On the other hand, the algorithm may become ineffective if too few neighbor solutions are tested. Please note, that before exploring a neighborhood, the solution for the exploration needs to be selected which takes non-negligible CPU time even with the use of ND-Tree. Thus the optimum strategy may be to test a randomly selected subset of neighbor solutions. Dubois-Lacoste et al. and Liefoghe et al. consider several options for partial exploration of the neighborhoods. Our approach is slightly different since we use the number of tested moves as a parameter of the method.

2.5. MPLS algorithm

The proposed method is summarized in Algorithm 3. To draw

Algorithm 3 MPLS.

Parameter $\downarrow$: Maximum running time T
 Parameter $\downarrow$: Number of neighborhood moves M
 Parameter $\uparrow$: $\mathcal{A}$: an initial Pareto archive

while total running time is lower than T **do**

 Use as the reference point y^0 the point composed of the maximum values of particular objectives in the current archive increased by 10% of the range of a given objective in this archive
 Draw at random a weight vector Λ defining a Chebycheff scalarizing function $s_\infty(\dots, y^0, \Lambda)$
 Find in $\mathcal{A}$ the solution x minimizing $s_\infty(\dots, y^0, \Lambda)$, i.e. call `MinimizeChebycheffScalarizingFunction` ($rn, s_\infty(\dots, y^0, \Lambda), \infty, x$) with rn being a root node of ND-Tree

for M times **do**

 Generate a random neighbor x' of x

if $y(x) \neq y(x')$ and $y(x) \neq y(x')$ **then**

`UpdateNDTree`($\mathcal{A} \uparrow, x' \downarrow$)

end if

end for

end while

where $\mathcal{N}(x)$ denotes the neighborhood of x and `UpdateNDTree`($\mathcal{A}$) updates the Pareto archive $\mathcal{A}$ using ND-Tree data structure.

a random weight vector we use the algorithm proposed in

Jaskiewicz (2002). Weighted Chebycheff scalarizing functions are applied to the objective values normalized on-line to the range $[0, 1]$ based on the ranges of the objectives in the current archive, which is in fact achieved by dividing each individual weight by the range of the corresponding objective.

Algorithm 3 defines the general scheme of MPLS. To apply it to a given problem a neighborhood structure has to be defined. The choice of the neighborhood structure may of course highly influence the performance of the algorithm.

3. Computational experiment

To test the effectiveness of the proposed algorithm we use two different combinatorial optimization problems. All algorithms were implemented in C++ sharing as much of the code as possible. The experiment has been run on an Intel Core i7-5500U CPU at 2.4 gigahertz. The source code, the instances, and the detailed numerical results are available on-line.¹

3.1. Multiobjective traveling salesperson problem

As one of the test problems we use the Multiobjective Traveling Salesperson Problem (MTSP) which is a typical benchmark problem for multiple objective metaheuristics.

Given a set $\{v_1, v_2, \dots, v_N\}$ of nodes and d costs $c_1(v_i, v_j) \dots c_d(v_i, v_j)$ between each pair of distinct nodes $\{v_i, v_j\}$, the multiobjective traveling salesperson problem (MTSP) consists of finding an order π of the nodes, minimizing the following costs ($k = 1, \dots, d$):

$$\text{“minimize” } y_k(\pi) = \sum_{i=1}^{N-1} c_k(v_{\pi(i)}, v_{\pi(i+1)}) + c_k(v_{\pi(N)}, v_{\pi(1)})$$

In the computational experiment, we used the symmetric multiobjective traveling salesperson problem with $c_k(v_i, v_j) = c_k(v_j, v_i)$ for $1 \leq i, j \leq N$. We used Euclidean instances with 100 and 200 nodes in which the costs of the edges correspond to the Euclidean distance between two points in a plane, and each objective corresponds to a different plane. For the purpose of this experiment, we generated 10 instances for each size and each number of objectives.

Following the successful methods proposed for the bi-objective case, we used a two-phase approach (Lust & Teghem, 2010). In the first phase we used Lin-Kernighan heuristic (Lin & Kernighan, 1973) to generate an initial Pareto archive for the second phase in which MPLS was run. Precisely we used the efficient implementation of this heuristic from the Concorde project (<http://www.math.uwaterloo.ca/tsp/concorde.html>).

We used the 2-edge exchange move in MPLS. Alike Lust and Jaskiewicz (2010) and Jaskiewicz and Lust (2017) we used the mechanisms of candidate moves to speed-up the calculations.

3.2. Comparison of the various versions of PLS

In the first experiment we compare MPLS to the standard PLS and to several other versions of PLS. Particular versions of PLS allow us to show not only that MPLS performs better than the standard PLS but also that each of the three new mechanisms used in MPLS indeed influences its effectiveness. The following methods are compared in this experiment:

- “MPLS 100” described by Algorithm 3 with 100 random moves tested in each neighborhood, which is a relatively good value of this parameter according to our preliminary experiments.

¹ <https://sites.google.com/view/mopls/mpls>

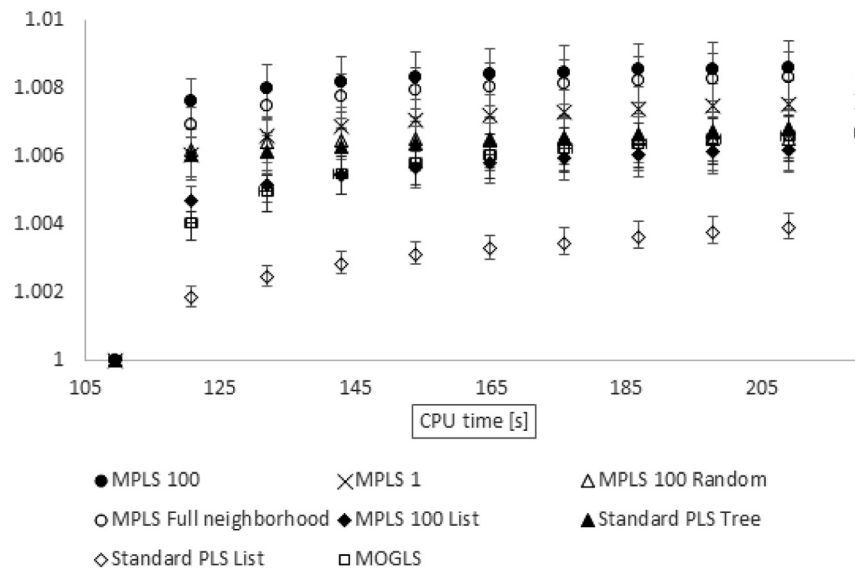


Fig. 3. Hypervolume indicator for 3-objective MTSP instances with 100 nodes.

- “MPLS full neighborhood” – the algorithm equivalent to “MPLS 100” but with the full exploration of each neighborhood. This algorithm allows us to show that the partial exploration of the neighborhoods improves the performance of MPLS.
- “MPLS 1” described by Algorithm 3 with just one random move tested in each neighborhood. We use this algorithm to show that too small number of tested moves deteriorates the effectiveness of MPLS.
- “MPLS 100 Random” – the algorithm equivalent to “MPLS 100” but with the random selection of solutions with uniform probability. This algorithm allows us to show that the selection of solutions based on the randomly selected weighted Chebycheff scalarizing functions improves the performance of MPLS.
- “MPLS 100 List” – the algorithm equivalent to “MPLS 100” but with the Pareto archive organized as a simple list. This algorithm allows us to show that the use of ND-Tree data structure improves the performance of MPLS.
- “Standard PLS tree” described by Algorithm 1 but using ND-Tree data structure. This algorithm allows us to show that the selection of solutions based on the randomly selected weighted Chebycheff scalarizing functions and the partial exploration of the neighborhoods improves the performance of MPLS.
- “Standard PLS List” described by Algorithm 1. This algorithm is the straightforward adaptation of the algorithm used in the bi-objective case to the many-objective case without any of the three new mechanisms used in MPLS.

The number of neighborhood moves M was set to 100 for all instances. It should not be considered, however, as the best value of this parameter in each case. We use this setting as a proof that the best results may be achieved with some intermediate approach between testing just one move or the whole neighborhood.

Please note, that in fact the standard PLS algorithm (see Algorithm 1) does not specify in which order set P is processed. In our experiment we processed set P in the natural order in which the solutions were added to this set.

In the first phase, we used 1000, 2000, and 3000 random weight vectors for $d = 3, 4, 5$, respectively. For each of the weight vectors the Lin–Kernighan heuristic was run optimizing a weighted sum of the objectives. Then, in the second phase MPLS/PLS was allowed to run for the maximum time equal to the running time of the first phase.

The results of this experiment are presented in Figs. 3–6. For each size and each number of objectives 10 different instances were used. As the quality measure we report the hypervolume indicator, but we calculated also the R indicator, i.e. average value of the weighted Chebycheff scalarizing functions Jaskiewicz (2002) (i.e. approximation of the expected value) with exactly the same conclusions. The detailed numerical results of the two quality indicators are available on-line.² Each point represents the average value and the range of the hypervolume indicator on 10 different instances obtained by a given method. The first point always corresponds to the Pareto archive obtained after the first phase. Since, we used different instances we normalize the values of the hypervolume such that it is equal to 1 after the first phase.

The reference points for the hypervolume were set to approximate nadir points of each instance with objective values multiplied by 1.5. To obtain an approximate nadir point for a given instance, each objective was optimized individually with either Lin–Kernighan heuristic (MTSP) or local search (MTSPWP). Then the worst obtained values of each objective were used as the coordinates of the approximate nadir point.

The reported running times include also the running times of the first phase. For each instance, the maximum running time in the second phase was equal to the running time of the first phase. The values of the quality indicators were calculated in time steps equal to 10% of the running time of the first phase. The running times were averaged over 10 runs in 10 instances. Since the variations of the running times of the first phase were relatively low, we report just the average times.

The main observations are:

- The relative differences between the methods grow with the growing number of objectives. In other words, the more objectives the more important are the three new mechanisms used in MPLS.
- Each of the three new mechanisms of MPLS is important for the final performance of the method. The comparison of “MPLS 100” to “MPLS 100 List” indicates that the mechanism with the highest influence on the effectiveness of MPLS is the use of ND-Tree data structure to update the Pareto archive and to select the best solution for a given scalarizing function. The compar-

² <https://sites.google.com/view/mopls/mpls>

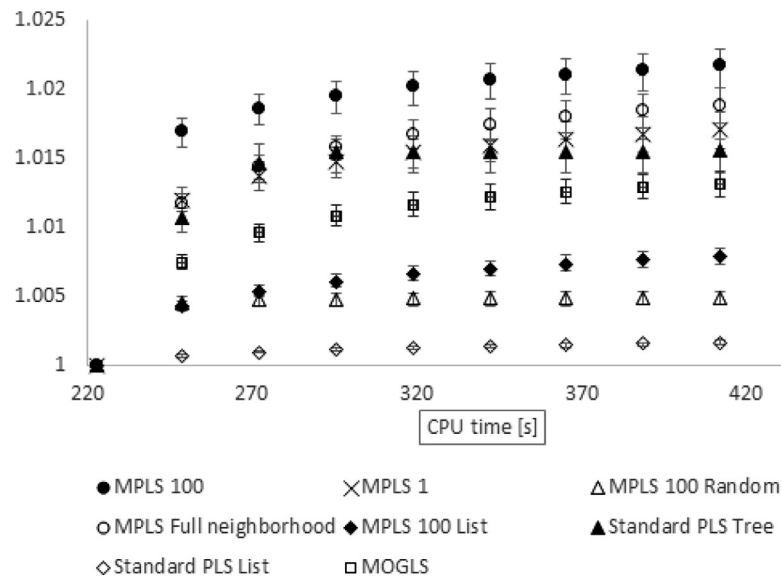


Fig. 4. Hypervolume indicator for 4-objective MTSP instances with 100 nodes.

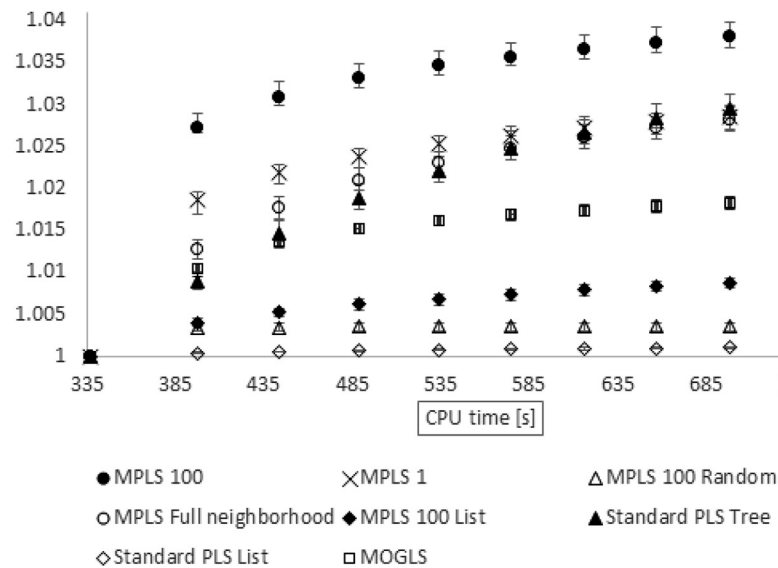


Fig. 5. Hypervolume indicator for 5-objective MTSP instances with 100 nodes.

ison of “MPLS 100” to “Standard PLS Tree” indicates that the second mechanism with the highest influence on the effectiveness of MPLS is the selection of solutions based on the randomly selected weighted Chebycheff scalarizing functions. Furthermore, the comparison to “MPLS 100 Random” indicates that the random selection of solutions with uniform probability is even worse than the selection mechanism used in the standard PLS. The comparison of “MPLS 100” to “MPLS Full neighborhood” and “MPLS 1” indicates that the partial exploration of the neighborhoods also substantially improves the performance of MPLS.

- “Standard PLS List”, i.e. the algorithm that uses none of the three new mechanisms used in MPLS is the worst algorithm in all cases.

In order to test the significance of the differences we used the non-parametric statistical test of Mann–Whitney (Ferguson, 1967). The level of risk of the test has been fixed to 5%. The final results of “MPLS 100” were significantly better than the final results of

all other methods in all cases except of “MPLS full neighborhood” with $d = 3$.

We have made some additional experiments with full MPLS using $1_{\neq}$ and $1_{\leq}$ stopping conditions described in Liefoghe et al. (2012). With these stopping conditions the exploration of a neighborhood was stopped after finding the first solution non-dominated with respect to the current one ($1_{\neq}$) or the first solution dominating the current one ($1_{\leq}$). These results are available on-line (see Section 3) but not included in this paper since they were very similar to other algorithms tested. $1_{\leq}$ version behaved very similarly to “MPLS full neighborhood”. This is because unlike Liefoghe et al. (2012) we initialize the archive with very high quality solutions. In result we observed that only very rarely a dominating solution was found in a neighborhood. Thus, in most cases the whole neighborhood was tested as in “MPLS full neighborhood”. On the other hand $1_{\neq}$ stopping condition resulted in testing very few moves (often just one) since a solution non-dominated with respect to the current one was quickly found. Again please note, that unlike Liefoghe et al. (2012) we use the

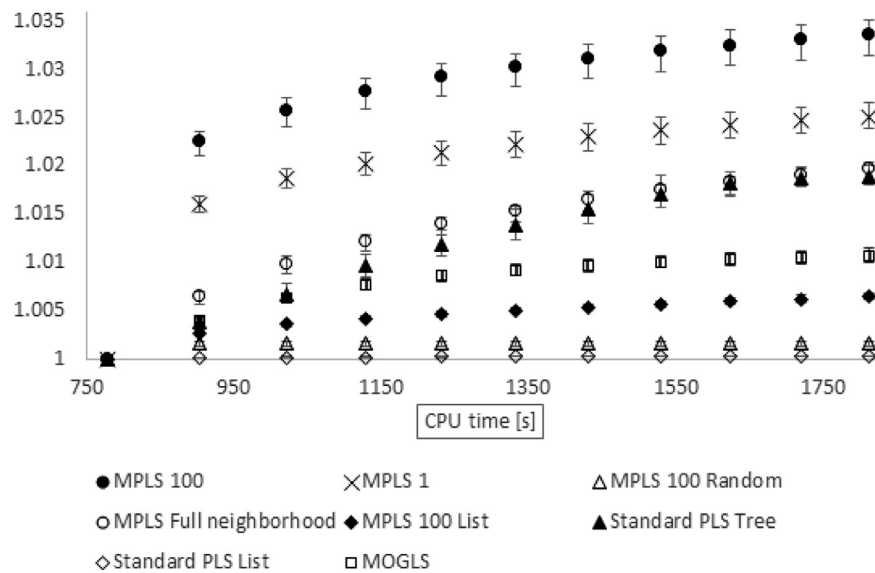


Fig. 6. Hypervolume indicator for 5-objective MTSP instances with 200 nodes.

concept of candidate moves that increases the chance of testing high quality moves only.

In addition, we compared MPLS to the Multiobjective Genetic Local Search (MOGLS) algorithm with path relinking (Jaskiewicz & Zielniewicz, 2009) for the multiobjective traveling salesperson problem. According to our knowledge the only methods reported in the literature that improved the results of this version of MOGLS are those using PLS. The initial archive of MOGLS was exactly the same as in the case of MPLS. The expected rank of recombined solutions was set to 20 based on preliminary experiments. MOGLS outperformed several variants of PLS/MPLS including “Standard PLS List” and “MPLS 100 Random” but was significantly outperformed by the full version of MPLS, i.e. “MPLS 100”.

3.3. MPLS with various number of runs of the Lin–Kernighan heuristic

In the above experiment we have shown that MPLS performs better than other versions of PLS. The question remains, however, whether it is competitive to Lin–Kernighan heuristic standalone applied alike in the initial phase. Since, in the bi-objective case the best results have been obtained combining the phases of the Lin–Kernighan heuristic and of PLS (Cornu et al., 2017; Jaskiewicz & Lust, 2017; Liangjun et al., 2014; Lust & Jaskiewicz, 2010; Lust & Teghem, 2010), we test whether MPLS can improve the results obtained with Lin–Kernighan heuristic standalone also in the many-objective case. Precisely, we ask the following question: Having some predefined CPU time, can we obtain the best result with the use of the Lin–Kernighan heuristic standalone, or by combining it with MPLS in two phases.

In the first phase, we used 2000, 4000, and 6000 as the maximum number of random weight vectors for $d = 3, 4, 5$, respectively. For each of the weight vectors the Lin–Kernighan heuristic was run optimizing a weighted sum of the objectives. The time needed for all runs of the Lin–Kernighan heuristic was then used as the maximum running time. Then, the number of Lin–Kernighan heuristic runs was reduced to 90%, ..., 10% and the remaining running time was allocated to MPLS. The numbers of weight vectors and thus the numbers of Lin–Kernighan heuristic runs were twice higher than in the previous experiment (see Section 3.2). Please note, however, that in the first experiment the same CPU time as in the first phase was then allocated to MPLS/PLS. In the second ex-

periment the number of weight vectors was gradually reduced and only the remaining time was allocated to MPLS/PLS. Thus, the total running time was approximately the same in both experiments.

The results are presented in Figs. 7–10. The values of the hypervolume are again normalized such that it is 1 after the first phase with the maximum number of the Lin–Kernighan heuristic runs. Please note, that for the lower numbers of the Lin–Kernighan heuristic runs the quality after the first phase is not shown since it was much lower and the figures would become incomprehensible.

The main observations are:

- In almost all cases, the results with the use of MPLS are better than with the use of the Lin–Kernighan heuristic standalone even with the lowest number of runs of the Lin–Kernighan heuristic. The only exception is the result for the 5-objective instances with 200 nodes with the smallest number of the Lin–Kernighan heuristic runs. This exception is caused by the fact that as observed in Jaskiewicz and Lust (2017) the number of Lin–Kernighan heuristic runs should grow with the size of an instance, while in our case, we used the same numbers for instances with 100 and 200 nodes in our experiment.
- The overall best results were obtained for 1000, 2400, 4200, and 4800 runs of the Lin–Kernighan heuristic, for $d = 3, 4, 5$ with 100 nodes and $d = 5$ with 200 nodes, respectively, i.e. the best results were obtained with 50% or less of the CPU time allocated to MPLS. In other words, alike in the bi-objective case the best results are obtained by properly balancing the CPU time allocated to the phases of the Lin–Kernighan heuristic runs and MLPS. Please note, that the Lin–Kernighan heuristic, and especially its efficient implementation from the Concorde project, is an advanced method for TSP, while MPLS uses the very basic 2-edge exchange move.
- The standard PLS algorithm was able to improve the results obtained with the Lin–Kernighan heuristic standalone only in the 3-objective case. In the 4- and 5-objective cases, the standard PLS algorithm was not able to improve the results obtained with the Lin–Kernighan heuristic standalone for any reduction of the number of runs in the first phase. Thus, the use of this algorithm in these cases would not be beneficial.

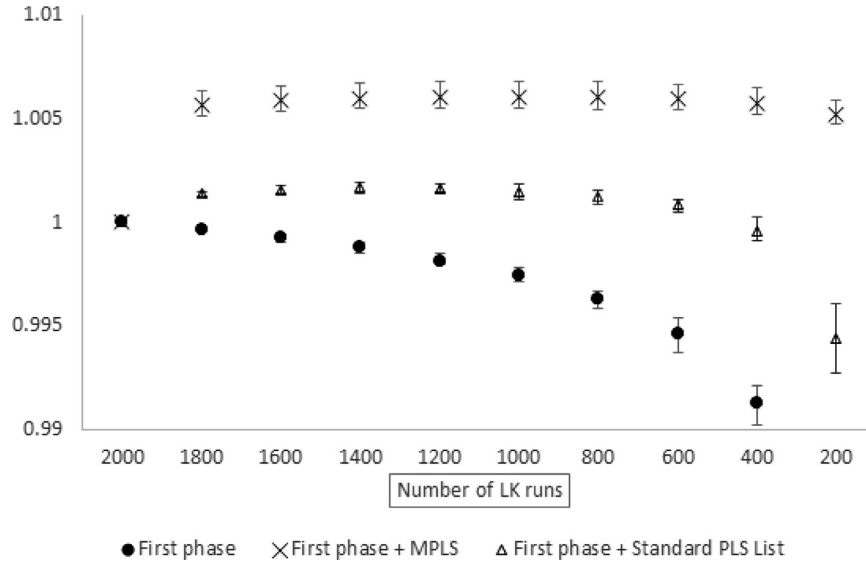


Fig. 7. Hypervolume indicator for 3-objective MTSP instances with 100 nodes with various numbers of the Lin–Kernighan heuristic runs and constant total CPU time.

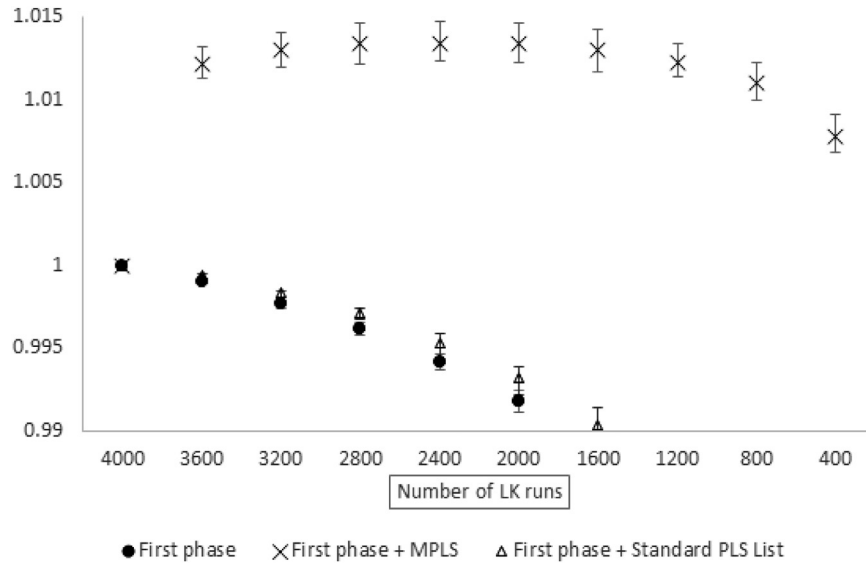


Fig. 8. Hypervolume indicator for 4-objective MTSP instances with 100 nodes with various numbers of the Lin–Kernighan heuristic runs and constant total CPU time.

3.4. Multiobjective traveling salesperson problem with profits

Given a set $\{v_1, \dots, v_N\}$ of nodes and d_1 costs $c_1(v_i, v_j) \dots c_{d_1}(v_i, v_j)$ between each pair of distinct nodes $\{v_i, v_j\}$, and d_2 profits $pr_1(v_i) \dots pr_{d_2}(v_i)$ associated with each node $\{v_i\}$, the multiobjective traveling salesperson problem with profits (MTSPWP) consists of choosing a subset of nodes SN and finding an order π of these nodes, optimizing the following objectives:

$$\text{“minimize” } y_k(\pi) = \sum_{i=1}^{|SN|-1} c_k(v_{\pi(i)}, v_{\pi(i+1)}) + c_k(v_{\pi(|SN|)}, v_{\pi(1)}), k = 1, \dots, d_1$$

$$\text{“maximize” } y_{d_1+l}(\pi) = \sum_{i=1}^{|SN|} pr_l(v_{\pi(i)}), l = 1, \dots, d_2$$

In other words, d_1 objectives correspond to the minimization of d_1 costs of the routes, while d_2 objectives correspond to the maximization of d_2 profits associated with the selected nodes.

We used the symmetric costs where $c_k(v_i, v_j) = c_k(v_j, v_i)$ for $1 \leq i, j \leq N$. For the purpose of this experiment we generated 10 instances with 200 nodes for each number of objectives. For $d = 3$, we used one cost objective and two profit objectives, for $d = 4$, we used two cost objectives and two profit objectives, and for $d = 5$, we used two cost objectives and three profit objectives. The cost objectives were generated using Euclidean distances, i.e. the costs of the edges correspond to the Euclidean distance between two points in a plane, and each objective corresponds to a different plane. Profits were drawn at random with a uniform distribution from a range $[0, 2000]$.

Please note, that despite of the similar names MTSPWP differs substantially from MTSP. In particular:

- The two problems have different solution spaces. In MTSPWP the solutions are defined by both a choice of the nodes, and an order of the selected nodes. In result, MTSPWP requires a different set of the neighborhood moves described below.
- MTSPWP exhibits a specific pattern of positive/negative correlations of objectives. The cost objectives are in general posi-

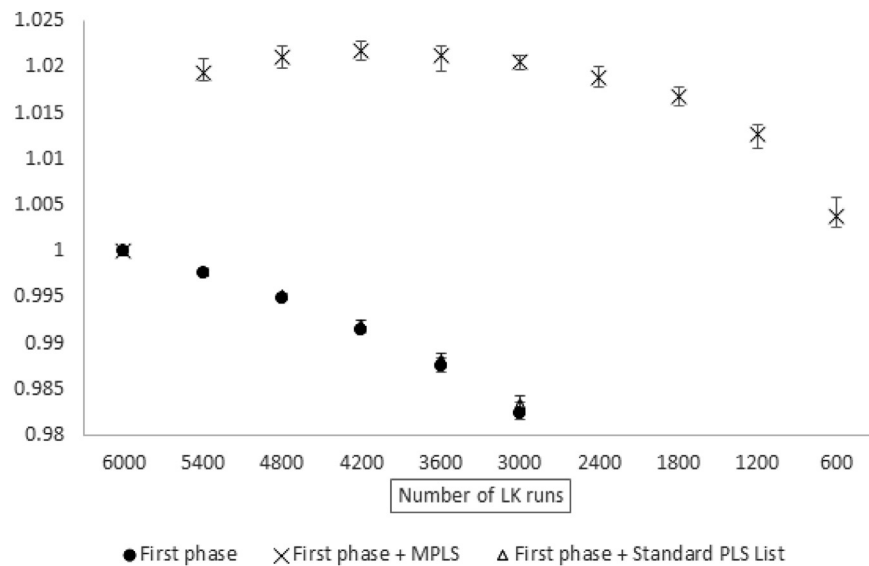


Fig. 9. Hypervolume indicator for 5-objective MTSP instances with 100 nodes with various numbers of the Lin–Kernighan heuristic runs and constant total CPU time.

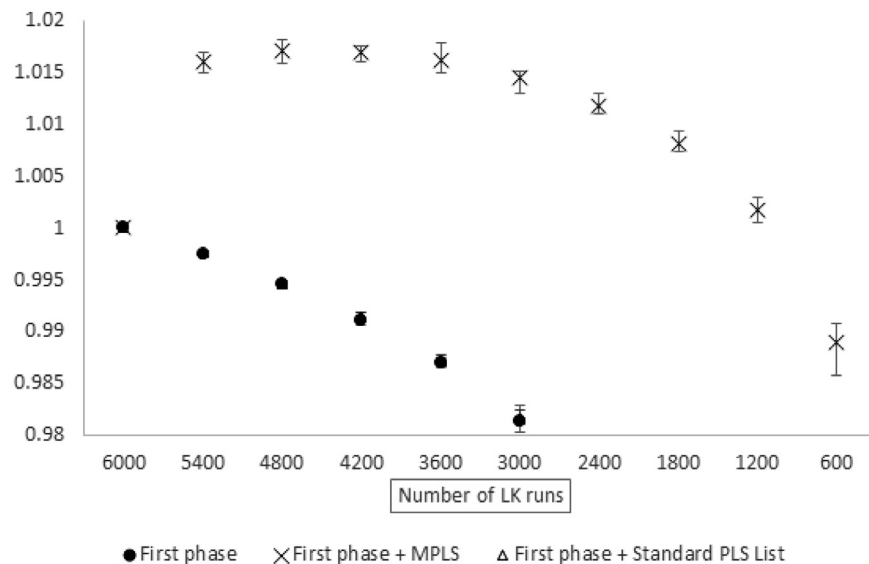


Fig. 10. Hypervolume indicator for 5-objective MTSP instances with 200 nodes with various numbers of the Lin–Kernighan heuristic runs and constant total CPU time.

tively correlated because all of them improve with reducing the number of selected nodes. The profit objectives also are in general positively correlated because all of them improve with the growing number of selected nodes. On the other hand, the two groups of objectives are negatively correlated for the same reasons.

- The cost and profit objectives are defined by heterogeneous mathematical formulas.

Because of the nature of MTSPWP, several types of moves were used to both select appropriate subset of nodes and optimize the route between these nodes (Jozefowiec, Glover, & Laguna, 2008):

- “2-edge exchange move” is the same move as in the case of MTSP.
- “Node delete move” removes one of the nodes from the current solution.
- “Node insert move” inserts one of the nodes that is not selected at a given position in the route.
- “Node exchange move” exchanges one of the nodes from the current solution for one of the nodes that is not selected.

The design of the experiments was similar as in the case of MTSP. In the first phase, instead of the Lin–Kernighan heuristic that cannot be applied in this case, we used the steepest local search algorithm that was testing all moves of all types before selecting the best neighbor. In MPLS the type of the move was drawn at random, and then the specific move was drawn. The steepest local search was applied to the optimization of the combined scalarizing functions being the weighted sums of the Chebycheff scalarizing function with weight 1 and the linear scalarizing function with weight 0.5, which was found the best option in the preliminary experiments. This version was found to perform better than the weighted sum used in the case of MTSP and the Lin–Kernighan heuristic. The steepest local search was then run for each of the randomly generated weight vectors.

We have performed the two experiments analogous like in the case of MTSP with same numbers of local search runs like the numbers of the Lin–Kernighan heuristic runs. Despite of the different nature of the two problems the results of the experiments for MTSPWP are very similar to the results for MTSP. Because of the limited space we present only the results for the 5-objective instances in Figs. 11 and 12.

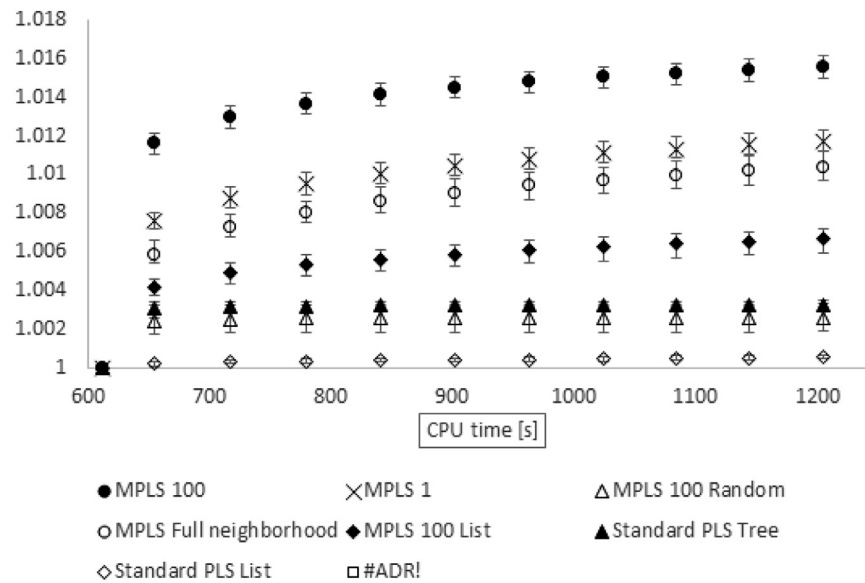


Fig. 11. Hypervolume indicator for 5-objective MTSPWP instances.

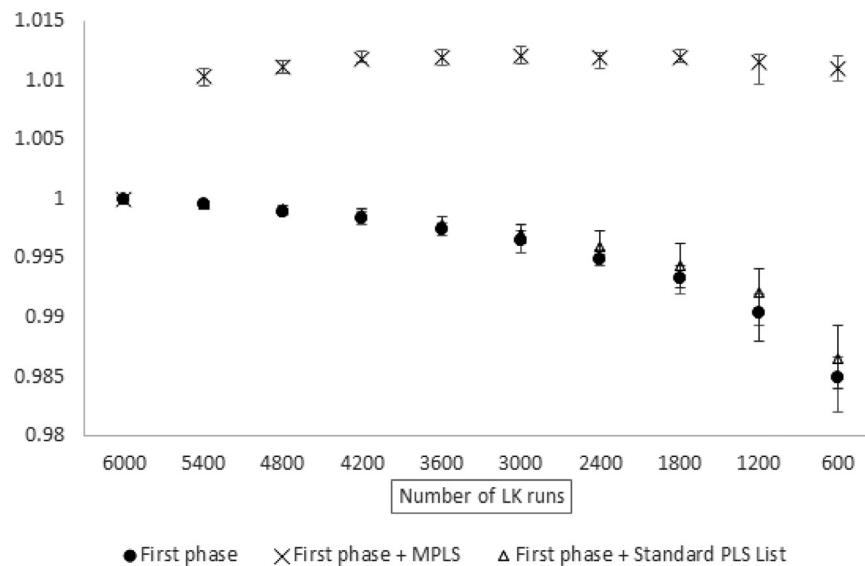


Fig. 12. Hypervolume indicator for 5-objective MTSPWP instances with various numbers of local search runs and constant total CPU time.

4. Related works

Pareto Local Search algorithm has been proposed by Angel et al. (2004), Paquete and Stutzle (2006) and Paquete et al. (2007). It has been successfully applied to the bi-objective knapsack (Lust & Teghem, 2012), the bi-objective traveling salesperson problem (TSP) (Cornu et al., 2017; Jaskiewicz & Lust, 2017; Liangjun et al., 2014; Lust & Jaskiewicz, 2010; Lust & Teghem, 2010), various bi-objective permutation flowshop problems (Dubois-Lacoste et al., 2011), the bi-objective set covering problem (Lust & Tuytens, 2014) and the risk-cost optimization for procurement planning (Mori et al., 2017). Some recent advancements in PLS involve: Pareto improvement of some dominated solutions, i.e. selection of a dominating solution from the neighborhood, until such improvements are no longer possible (Inja, Kooijman, de Waard, Roijers, & Whiteson, 2014); the use of larger perturbations of solutions (Drugan & Thierens, 2012; Liangjun et al., 2014), the penalization of some features exhibited by many solutions in the archive (Alsheddy & Tsang, 2010), and the use of the data perturbations

(Cornu et al., 2017). However, none of these proposals focuses on the many-objective optimization.

One of the main ideas of MPLS is to improve the anytime behavior of the algorithm, since obtaining the locally optimal Pareto archive is usually prohibitively time consuming in the many-objective case. An anytime PLS algorithm has been proposed by Dubois-Lacoste et al. (2011, 2015). Some similar concepts have been analyzed by Liefoghe et al. (2012). Anytime PLS algorithm is, however, dedicated to the bi-objective case, and the authors define many-objective optimization as a direction for further research. At a general level some mechanisms used in MPLS are motivated by these works, but the specific techniques are very different. For example Dubois-Lacoste et al. use the optimistic hypervolume to select the promising solutions, which cannot be directly applied in the many-objective case.

Liangjun et al. (2014) used PLS to solve the multiobjective knapsack problem instances with up to 4 objectives. Cornu et al. (2017) applied PLS to the bi- and 3-objective instances of TSP. These papers do not contain, however, information whether any

specific mechanisms for the many-objective case, other than the reduction of the time allocated to PLS, were used.

The idea of using the scalarizing functions with the randomly generated weight vectors has been proposed in some multiobjective evolutionary algorithms, e.g. by Ishibuchi and Murata (1998), Jaskiewicz (2002), Jaskiewicz and Zielniewicz (2009). The scalarizing functions were, however, used in a very different way to select solutions for recombination and to optimize the scalarizing functions with the single objective local search.

The multiobjective traveling salesperson problem is often used as a benchmark problem for the multiobjective metaheuristics (Jaskiewicz, 2002). As it was already mentioned, the state-of-the-art methods for the bi-objective TSP involve the use of the Lin-Kernighan heuristic and Pareto Local Search (Cornu et al., 2017; Jaskiewicz & Lust, 2017; Liangjun et al., 2014; Lust & Jaskiewicz, 2010; Lust & Teghem, 2010).

The traveling salesperson problem with profits is a well-known combinatorial optimization problem, however, it is usually considered as a single objective problem with a single cost and a single profit aggregated to one objective with a weighted sum approach (Feillet, Dejax, & Gendreau, 2005). There are relatively few results for the bi-objective case (Bérubé, Gendreau, & Potvin, 2009; Jozefowicz et al., 2008; Labadie, Melechovsky, & Prins, 2014; Zhu, Suzuki, & Boonma, 2011). To our knowledge, multiobjective metaheuristics have not been applied yet to the multiobjective version of the traveling salesperson problem with profits with more than one cost and/or profit.

5. Conclusions and directions for further research

We have presented a new Many-Objective Pareto Local Search algorithm. The algorithm has been tested on two different multiobjective combinatorial optimization problems showing its effectiveness. We have also shown that each of the three new mechanisms used in MPLS significantly contributes to its effectiveness. Furthermore, the importance of the new mechanisms in relation to the standard PLS grows with the growing number of objectives. In particular, in the 4- and 5-objective cases, standard PLS without any of the new mechanisms was not able to improve the results obtained with the Lin-Kernighan heuristic (MTSP) or local search (MTSPWP) standalone for any reduction of the number of runs in the first phase. On the other hand, MPLS was able to improve these results in almost all cases tested in the experiment.

Since the standard PLS algorithm proved its high effectiveness in the bi-objective case, we believe that MPLS could be a very useful element in the toolbox for the general case of the multiobjective combinatorial optimization, especially when combined with other methods being very effective in the search towards the Pareto front. In fact, we expect that alike the standard PLS in the bi-objective case, MPLS could become a typical component of the state-of-the-art algorithms for the many-objective combinatorial optimization.

The applications of MPLS to various benchmark and real-life problems, and combination with other multiobjective metaheuristics is a natural direction for further research. In particular MPLS could be combined with such concepts as the use of larger perturbation of solutions (Liangjun et al., 2014) or data perturbations (Cornu et al., 2017).

MPLS introduces a new parameter, i.e. the number of the neighborhood moves. At present we do not have any guidelines for setting this parameter. Development of a mechanism for automatic adaptation of this parameter is an interesting direction for further research. In particular, the efficiency of the partial exploration of the neighborhood may be problem-dependent, and for some problems full exploration of the neighborhood may be still necessary.

MPLS generates a large number of potentially Pareto-optimal solutions. The maximum size of the Pareto archive in our experiment exceeded 5 millions of solutions. The final goal of the decision maker is to select the single solution being the best according to his/her preferences. If no information about these preferences is available, a multiobjective metaheuristic should generate a good solution for any possible preferences. However, often some partial preference information is known in advance or may be gathered during the run of the method. Thus, another interesting direction for further research is the incorporation of some partial preference information in order to limit the region of the Pareto front searched by MPLS.

Acknowledgment

This research was funded by the Polish National Science Center, grant no. UMO-2013/11/B/ST6/01075.

References

- Alsheddy, A., & Tsang, E. E. P. K. (2010). Guided Pareto local search based frameworks for biobjective optimization. In *Proceedings of the 2010 IEEE congress on evolutionary computation* (pp. 1–8). doi:10.1109/CEC.2010.5585983.
- Angel, E., Bampis, E., & Gourvès, L. (2004). A dynasearch neighborhood for the bi-criteria traveling salesman problem. In X. Gandibleux, M. Sevaux, K. Sörensen, & V. T'kindt (Eds.), *Metaheuristics for multiobjective optimisation* (pp. 153–176). Berlin, Heidelberg: Springer. doi:10.1007/978-3-642-17144-4_6.
- Bérubé, J.-F., Gendreau, M., & Potvin, J.-Y. (2009). An exact e-constraint method for bi-objective combinatorial optimization problems: Application to the traveling salesman problem with profits. *European Journal of Operational Research*, 194(1), 39–50. doi:10.1016/j.ejor.2007.12.014.
- Chand, S., & Wagner, M. (2015). Evolutionary many-objective optimization: A quick-start guide. *Surveys in Operations Research and Management Science*, 20(2), 35–42. doi:10.1016/j.sorms.2015.08.001.
- Cornu, M., Cazenave, T., & Vanderpooten, D. (2017). Perturbed decomposition algorithm applied to the multi-objective traveling salesman problem. *Computers & Operations Research*, 79, 314–330. doi:10.1016/j.cor.2016.04.025.
- Drugan, M. M., & Thierens, D. (2012). Stochastic Pareto local search: Pareto neighbourhood exploration and perturbation strategies. *Journal of Heuristics*, 18(5), 727–766. doi:10.1007/s10732-012-9205-7.
- Dubois-Lacoste, J., López-Ibáñez, M., & Stützle, T. (2011). A hybrid TP+PLS algorithm for bi-objective flow-shop scheduling problems. *Computers & Operations Research*, 38(8), 1219–1236. doi:10.1016/j.cor.2010.10.008.
- Dubois-Lacoste, J., López-Ibáñez, M., & Stützle, T. (2015). Anytime Pareto local search. *European Journal of Operational Research*, 243(2), 369–385. doi:10.1016/j.ejor.2014.10.062.
- Feillet, D., Dejax, P., & Gendreau, M. (2005). Traveling salesman problems with profits. *Transportation Science*, 39(2), 188–205. doi:10.1287/trsc.1030.0079.
- Ferguson, T. (1967). *Mathematical statistics, a decision theoretic approach*. New-York and London: Academic Press.
- Inja, M., Kooijman, C., de Waard, M., Roijers, D. M., & Whiteson, S. (2014). Queued Pareto local search for multi-objective optimization. In T. Bartz-Beielstein, J. Branke, B. Filipić, & J. Smith (Eds.), *Proceedings of the thirteenth international conference on parallel problem solving from nature – PPSN XIII* (pp. 589–599). Ljubljana, Slovenia: September 13–17, 2014. Cham: Springer International Publishing. doi:10.1007/978-3-319-10762-2_58.
- Ishibuchi, H., & Murata, T. (1998). A multi-objective genetic local search algorithm and its application to flowshop scheduling. *IEEE Transactions on Systems Man and Cybernetics Part C – Applications and Reviews*, 28(3), 392–403. doi:10.1109/5326.704576.
- Jaskiewicz, A. (2002). Genetic local search for multi-objective combinatorial optimization. *European Journal of Operational Research*, 137(1), 50–71. doi:10.1016/S0377-2217(01)00104-7.
- Jaskiewicz, A., & Lust, T. (2017). Proper balance between search towards and along Pareto front: biobjective TSP case study. *Annals of Operations Research*, 254, 111–130. doi:10.1007/s10479-017-2415-5.
- Jaskiewicz, A., & Lust, T. (2018). ND-Tree-based update: A fast algorithm for the dynamic non-dominance problem. *IEEE Transactions on Evolutionary Computation*. in press. doi:10.1109/TEVC.2018.2799684.
- Jaskiewicz, A., & Zielniewicz, P. (2009). Pareto memetic algorithm with path relinking for bi-objective traveling salesperson problem. *European Journal of Operational Research*, 193(3), 885–890. doi:10.1016/j.ejor.2007.10.054.
- Jozefowicz, N., Glover, F., & Laguna, M. (2008). Multi-objective meta-heuristics for the traveling salesman problem with profits. *Journal of Mathematical Modelling and Algorithms*, 7(2), 177–195. doi:10.1007/s10852-008-9080-2.
- Labadie, N., Melechovsky, J., & Prins, C. (2014). An evolutionary algorithm with path relinking for a bi-objective multiple traveling salesman problem with profits. In L. Benyoucef, J. C. Hennet, & M. K. Tiwari (Eds.), *Applications of multi-criteria and game theory approaches: manufacturing and logistics*. In Springer series in advanced manufacturing (pp. 195–223). Berlin: Springer-Verlag. doi:10.1007/978-1-4471-5295-8_10.

- Liangjun, K., Qingfu, Z., & Battiti, R. (2014). Hybridization of decomposition and local search for multiobjective optimization. *IEEE Transactions on Cybernetics*, 44(10), 1808–1820. doi:[10.1109/TCYB.2013.2295886](https://doi.org/10.1109/TCYB.2013.2295886).
- Liefooghe, A., Humeau, J., Mesmoudi, S., Jourdan, L., & Talbi, E.-G. (2012). On dominance-based multiobjective local search: Design, implementation and experimental analysis on scheduling and traveling salesman problems. *Journal of Heuristics*, 18(2), 317–352. doi:[10.1007/s10732-011-9181-3](https://doi.org/10.1007/s10732-011-9181-3).
- Lin, S., & Kernighan, B. (1973). An effective heuristic algorithm for the traveling-salesman problem. *Operations Research*, 21, 498–516.
- Lust, T., & Jaskiewicz, A. (2010). Speed-up techniques for solving large-scale biobjective TSP. *Computers & Operations Research*, 37(3), 521–533. doi:[10.1016/j.cor.2009.01.005](https://doi.org/10.1016/j.cor.2009.01.005).
- Lust, T., & Teghem, J. (2010). Two-phase Pareto local search for the biobjective traveling salesman problem. *Journal of Heuristics*, 16(3), 475–510. doi:[10.1007/s10732-009-9103-9](https://doi.org/10.1007/s10732-009-9103-9).
- Lust, T., & Teghem, J. (2012). The multiobjective multidimensional knapsack problem: A survey and a new approach. *International Transactions in Operational Research*, 19(4), 495–520. doi:[10.1111/j.1475-3995.2011.00840.x](https://doi.org/10.1111/j.1475-3995.2011.00840.x).
- Lust, T., & Tuytens, D. (2014). Variable and large neighborhood search to solve the multiobjective set covering problem. *Journal of Heuristics*, 20(2), 165–188. doi:[10.1007/s10732-013-9236-8](https://doi.org/10.1007/s10732-013-9236-8).
- Mori, M., Kobayashi, R., Samejima, M., & Komoda, N. (2017). Risk-cost optimization for procurement planning in multi-tier supply chain by Pareto local search with relaxed acceptance criterion. *European Journal of Operational Research*, 261(1), 88–96. doi:[10.1016/j.ejor.2017.01.028](https://doi.org/10.1016/j.ejor.2017.01.028).
- Paquete, L., Schiavinotto, T., & Stuetzle, T. (2007). On local optima in multiobjective combinatorial optimization problems. *Annals of Operations Research*, 156(1), 83–97. doi:[10.1007/s10479-007-0230-0](https://doi.org/10.1007/s10479-007-0230-0).
- Paquete, L., & Stuetzle, T. (2006). A study of stochastic local search algorithms for the biobjective QAP with correlated flow matrices. *European Journal of Operational Research*, 169(3), 943–959. doi:[10.1016/j.ejor.2004.08.024](https://doi.org/10.1016/j.ejor.2004.08.024).
- Steuer, R. (1986). *Multiple criteria optimization: Theory, computation and applications*. New-York: John Wiley & Sons.
- Talbi, E.-G., Basseur, M., Nebro, A. J., & Alba, E. (2012). Multi-objective optimization using metaheuristics: non-standard algorithms. *International Transactions in Operational Research*, 19(1–2), 283–305. doi:[10.1111/j.1475-3995.2011.00808.x](https://doi.org/10.1111/j.1475-3995.2011.00808.x).
- Yu, G. (2013). *Industrial applications of combinatorial optimization*: Vol. 16. Springer Science & Business Media.
- Zhu, B., Suzuki, J., & Boonma, P. (2011). Evaluating the alpha-dominance operator in multiobjective optimization for the probabilistic traveling salesman problem with profits. In E. Kita (Ed.), *Evolutionary algorithms* (pp. 271–294). Janjeza Trdine99, Rijeka, 51000, Croatia: INTECH Europe.
- Zitzler, E., Thiele, L., Laumanns, M., Fonseca, C., & da Fonseca, V. (2003). Performance assessment of multiobjective optimizers: An analysis and review. *IEEE Transactions on Evolutionary Computation*, 7(2), 117–132. doi:[10.1109/TEVC.2003.810758](https://doi.org/10.1109/TEVC.2003.810758).